Project Title & Problem Statement

AsOf

A policy time machine and audit trail for AI assistants

Agentic AI: LangGraph agents on IBM® Db2®

What did the policy say on that date, and what did our AI tell the customer?

IBM Db2 12.1.5 | LangChain + LangGraph | Open source, Apache-2.0 | Two domains, plus one written by another AI

Jay Malhotra | solo entry | Final submission

github.com/JustJay7/asof

Unstop Round 1 items

Where each Round 1 item is covered

Round 1 item	Slides
01 **Project Title & Problem Statement**	01 03
02 **Proposed Solution & Key Features**	04 05
03 **Selected Agentic AI Framework**	06
04 **Db2 Connector & Integration Approach**	07 08 09
05 **Architecture / Workflow Overview**	10 11 12
06 **Expected Impact**	13 14 15

Scored in Round 1 on

Innovation
04 05

Feasibility
13 15

Db2 integration approach
07 08 09

Architecture
10 11 12

Built with

IBM Db2 · langchain · langgraph · dbt · Apache Airflow · Langflow · SQLGlot

IBM® and Db2® are trademarks of International Business Machines Corporation, registered in many jurisdictions worldwide. LangChain and LangGraph logos are shown under LangChain's brand guidelines. dbt and the dbt logo are trademarks of dbt Labs, LLC; AsOf is not affiliated with dbt Labs. Apache Airflow, Apache, Airflow and the Airflow logo are either registered trademarks or trademarks of The Apache Software Foundation; no endorsement is implied. Db2, Langflow and SQLGlot appear as text because their owners give no permission for logo use.

Project Title & Problem Statement

Companies own what their AI assistant says

Moffatt v. Air Canada

2024 BCCRT 149, decided 14 Feb 2024

The airline's website chatbot told a grieving customer he could claim a bereavement fare after travelling. The airline's own policy said he could not. The tribunal held the airline liable and rejected the argument that the chatbot was responsible for its own words.

CA$812

ordered in damages, interest and fees, over one chatbot answer

The company owns every statement

The tribunal rejected the defence that the correct policy was published on another page.

Policies change, answers drift

Refund windows, fees and eligibility rules change. Bots, cached FAQ answers and help articles keep repeating the old version.

No evidence when it counts

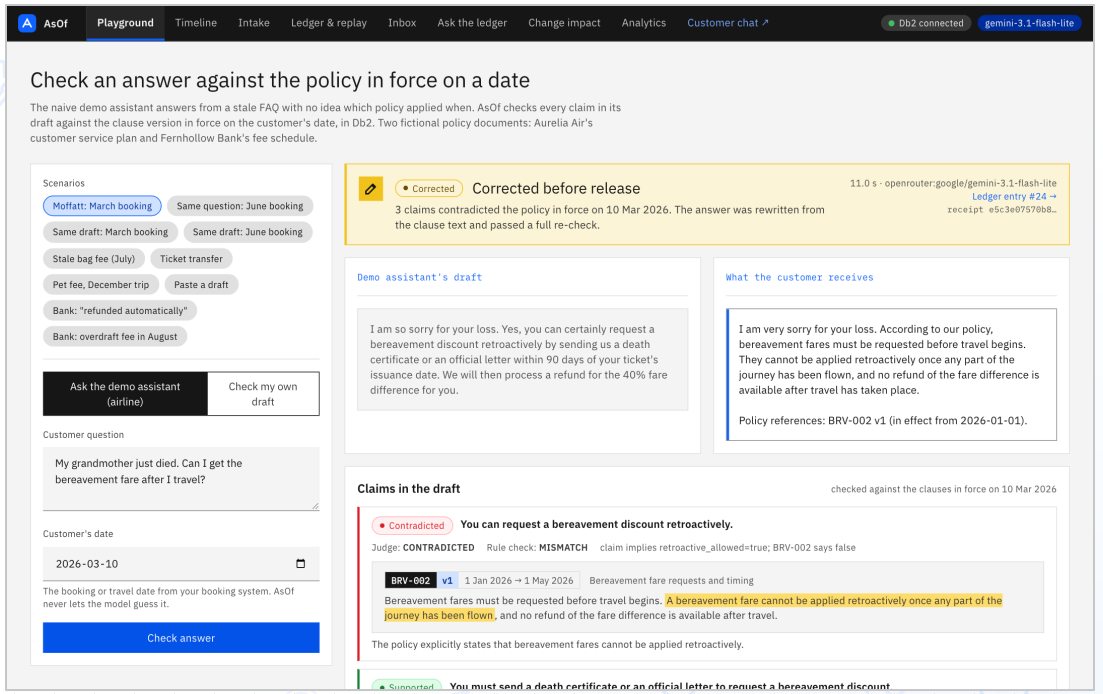
In a dispute, teams cannot show which policy version an answer relied on, or what their systems knew at that moment.

Proposed Solution & Key Features

AsOf: the policy in force on the customer's date

> *Given an AI-generated answer and a relevant date, AsOf finds the applicable policy version, flags claims that contradict it, stops unsupported answers from going out, and keeps the evidence needed to review that decision later.*

- **As-of lookup.** Db2 returns the clause versions in force on the customer's date.
- **Claim check.** LLM judge, verbatim quotes, rules on rule_json, and every number must reach a claim.
- **Decision.** PASS with citations, CORRECTED and re-checked, or ESCALATED. Fails closed.
- **Audit ledger.** Hash-chained Db2 record of every decision, replayable.
- **Change impact.** Stale answers get checked correction notices; what AsOf holds goes to a specialist.



The playground on a March booking: the demo assistant's retroactive promise is corrected from clause BRV-002 v1 before it reaches the customer.

Checked against

BRV-002 | v1 | in force 1 Jan until 1 May 2026 | CORRECTED

Proposed Solution & Key Features

Innovation: adding time to answer checking

> AsOf does not claim to have invented policy checking. What is new is time: what was true on date X, what our database knew at time T, and which past answers a change makes stale.

Closest existing tool	What it does	What AsOf adds
Amazon Bedrock Automated Reasoning checks	Validates answers against policies, suggests corrections, logs results	Effective-dated versions, replay of what the database knew, change impact, inside your own Db2, open source
NVIDIA NeMo Guardrails fact-checking rails	Scores an answer against retrieved chunks	Policy versions and dates, plus deterministic checks on structured rule parameters
IBM watsonx.governance	RAG quality metrics such as faithfulness	Per-answer evidence: which clause version, quoted, in a tamper-evident record

Effective-dated

Every clause version carries the dates it applied.

BRV-002 v2 in force from 1 May 2026

Bitemporal replay

FOR SYSTEM_TIME AS OF shows what the database held at answer time.

```
FOR SYSTEM_TIME AS OF ?
```

Change impact

A policy change lists the past answers and articles it makes stale.

STALE HISTORICALLY VALID

Selected Agentic AI Framework

Agentic AI: a LangGraph check, two agents and a router, built

langgraph

BUILT

- The check is a graph with fixed edges: extract, retrieve, judge, investigate, decide, rewrite
- Investigate: an agent whose tools query Db2 (as-of search, clause versions), at most 3 calls
- Change-impact agent: checked fixes for stale help articles and correction notices for customers
- Router: Db2's vector vote picks the document; close calls go to a measured agent or a person

langchain and IBM's tools

BUILT

SQLGlot

- Ask the ledger: a SQL chain over SQLAlchemy (ibm_db_sa), every query gated by SQLGlot's Db2 dialect
- langchain-db2 DB2VS, a custom retriever, a drop-in Runnable, and an MCP server for any agent

dbt | Apache Airflow | **Langflow**

- dbt™ marts (ibm-dbt-db2), an Apache Airflow® nightly audit (IBM Db2 provider), a Langflow component

Safety properties no agent step can bypass

- Fixed graph edges
- Rules override the model
- Every number must reach a claim
- Full re-check before release
- Any error means ESCALATED

Apache Airflow, Apache, Airflow and the Airflow logo are either registered trademarks or trademarks of The Apache Software Foundation; no endorsement is implied. LangChain and LangGraph logos are shown under LangChain's brand guidelines.

Db2 Connector & Integration Approach

Db2 integration: the database does the hard work

Bitemporal tables

Business and system time with a history table; FOR PORTION OF splits a version.

VECTOR and VANN

VECTOR(384) with VECTOR_DISTANCE in the temporal query; a native vector index, benchmarked.

SQL

SQL PL procedures

Twenty-two: a SHA-256 chain, change impact, a rolled-back preview, notices, intake, backtests.

In-database AI

CREATE EXTERNAL MODEL (OpenRouter or watsonx.ai) and TO_EMBEDDING, inside Db2.

JSON

SQL/JSON

JSON_EXISTS on every rule; JSON_TABLE in views and the citation shredder.

RCAC and workloads

Row permissions per domain, column masks, and a WLM workload that stops runaway AI-written SQL.

Partitions and MQTs

Monthly range partitions; REFRESH IMMEDIATE MQTs behind analytics, benchmarked at 1M rows.

Db2 enforces the release

A trigger refuses any released claim whose clause was not in force that day or whose quote is missing.

Connectors: ibm_db with bound parameters, langchain-db2 DB2VS, SQLAlchemy with ibm_db_sa, langchain-ibm, IBM's dbt adapter, Airflow provider and lfx-ibm.

Proof: python -m asof.db2check runs 34 of 34 Db2 feature checks; CI runs them, the full suite and an online BACKUP and RESTORE drill on IBM's stock image.

One query answers: what applied on that date?

```
SELECT c.CLAUSE_ID, c.VERSION_LABEL, c.CLAUSE_TEXT,
       c.RULE_JSON,
       VECTOR_DISTANCE(c.EMBEDDING,
         VECTOR(?, 384, FLOAT32), COSINE) AS DIST
FROM   POLICY_CLAUSE
       FOR BUSINESS_TIME AS OF ? AS c
WHERE  c.DOC_ID = ?
ORDER  BY DIST
FETCH  EXACT FIRST ? ROWS ONLY
```

Temporal filter and vector ranking in one statement. Every value is a bound parameter.

Example: raise the pet fee to $125 from 1 Dec 2026

PET-001	v1	in force 1 Jan until 1 Dec 2026
PET-001	v2	in force from 1 Dec 2026, $125

Nov answer: still valid | Dec answer: stale | Help article: review

Replay

Add FOR SYSTEM_TIME AS OF the answer's timestamp and Db2 returns exactly what it held when the bot answered, even if the policy was edited later.

Change impact

APPLY_POLICY_CHANGE updates FOR PORTION OF BUSINESS_TIME. CHANGE_IMPACT then lists each past answer as STALE or HISTORICALLY VALID, and each help article that needs review.

Db2 Connector & Integration Approach

What building AsOf taught us about Db2

A Db2 defect, with a repro

A forced vector-index scan under a date predicate fails with SQL0901N once the search list passes 200. One script reproduces it on IBM's stock image.

No vector index under time

Under FOR BUSINESS_TIME AS OF the optimizer never picked the vector index, even at 100,000 versions. AsOf keeps FETCH EXACT: 84 ms.

FETCH FIRST is approximate

Once a vector index exists, a plain FETCH FIRST reads it, and a ? for k never does. So the check says FETCH EXACT.

SQL

Three of six metrics index

CREATE VECTOR INDEX takes COSINE, EUCLIDEAN and EUCLIDEAN_SQUARED for FLOAT32; INT8 refuses COSINE.

JSON

JSON_TABLE: strict $ only

Its row path accepts only 'strict $', so the release trigger reads each claim by its position in the array.

WLM on Community Edition

WLM thresholds work on 12.1.5 CE. Cost estimates for cross joins of views swung from 17 to 10^8 timerons, so the time limit is what stops them.

Each finding is pinned by a test or a plan file in the repo (README: What building AsOf taught us about Db2).

Architecture / Workflow Overview

How one answer flows through AsOf

01 Answer in

Draft, question and the customer's date from the calling system. The model never picks the date.

→

02 Extract claims

Structured output: up to 8 atomic policy claims.

→

03 Retrieve as of date

One Db2 query: FOR BUSINESS_TIME AS OF plus VECTOR_DISTANCE.

04 Judge and check

One batched judge call; an agent searches Db2 if a claim matched nothing. Verbatim quotes; rule_json overrides.

→

05 Decide

PASS, CORRECTED after a full re-check, or ESCALATED. Unchecked numbers or errors: held.

→

06 Record

APPEND_LEDGER writes a hash-chained row; a Db2 trigger refuses a release the policy in force does not back.

10 Mar 2026	*“You can request the bereavement fare after you travel”*	BRV-002 v1 in force 1 Jan until 1 May 2026 request before travel	CORRECTED
2 Jun 2026	*“You can request the bereavement fare after you travel”*	BRV-002 v2 in force from 1 May 2026 retroactive within 90 days	PASS

Same question, same words, different dates: the answer is right or wrong depending on which version was in force.

Architecture

Any AI assistant

A naive demo bot (DB2VS FAQ store), a customer chat page, agents via MCP

AsOf API (FastAPI)

Strict Pydantic validation, rate limit, security headers

Web UI

Playground, timeline, intake, ledger, inbox, Ask the ledger, impact, backtest, analytics

Check pipeline

langgraph langchain

LangGraph check graph with an investigating agent (Db2 tools), deterministic checks, a change-impact agent for fixes and notices, a router, and a SQL chain for auditors. Policy PDFs become checked drafts. Models: OpenRouter gemini-3.1-flash-lite or IBM watsonx.ai, falling back to local Ollama per call. Embeddings: MiniLM, local or in Db2.

IBM Db2 12.1.5, schema ASOF

Users: asofapp, asofedit, asofrev, asofread, asofmart, and an analyst and a specialist per domain

Policy

POLICY_DOC, POLICY_CLAUSE + history: bitemporal, VECTOR(384), RULE_JSON; PDF drafts

Ledger

ANSWER_LEDGER (monthly partitions), CLAIM_CITATION, CHAIN_HEAD: append-only, hash-chained

Content

PUBLISHED_ANSWER (help centre), FAQ vector store (DB2VS)

Routines and controls

22 SQL PL procedures, a release trigger, row permissions and column masks, a WLM workload, MQTs

Security and trust by design

Fail closed

Any error, timeout or invalid model output becomes ESCALATED, and Db2 refuses to record a release the policy does not back.

Least privilege

Writes only through procedures. Row permissions give each domain's users their own rows; a workload stops runaway SQL.

No string-built SQL

Bound parameters, enforced by an AST build check. Generated SQL runs only after a SQLGlot gate.

Prompt injection

Delimited data, strict schemas, rule overrides, a coverage check behind the extractor, and verbatim checks on policy PDFs.

Redaction and masks

Personal data redacted before storage; Db2 column masks on ledger text.

Tamper evidence

Triggers block UPDATE and DELETE; VERIFY_CHAIN finds the first altered row; receipts catch a rebuilt ledger.

Expected Impact

Feasibility: built, tested and measured

1 of 121

wrong drafts released across nine frozen splits; 4 of 70 correct ones stopped

30 of 40

right calls on a railway policy another AI wrote to beat it; a plain LLM judge: 6 of 40

6 of 16

date-sensitive wrong drafts released if judged by today's policy (AsOf: 0)

1311

tests (80% line coverage, mypy), CI on IBM's stock Db2 image

Frozen splits, gemini-3.1-flash-lite, percent

Split	False allow	False block	Ideal decisions
Airline (40)	0.0%	7.1%	97.5%
Fewer holds (33)	0.0%	18.2%	90.9%
Premise, pinned first (20)	0.0%	0.0%	90.0%
Lists, pinned first (16)	0.0%	14.3%	87.5%
Situation 2 (12)	14.3%	0.0%	75.0%
Situation (12)	0.0%	0.0%	75.0%
Bank (26)	0.0%	0.0%	100.0%
Blind bank (22)	0.0%	0.0%	100.0%
Extractor attacks (10)	0.0%	0.0%	100.0%

Latest run of each split, after Db2 began enforcing releases. All 191 cases were pinned before their split's first run; the five that never motivated a fix: 0 of 65 released. On the unseen railway policy both AsOf and the plain judge let 2 of 25 wrong drafts through.

False allow: a wrong draft released as written. False block: a correct draft rewritten or held. Ideal: the labelled decision exactly.

Expected impact

For CX platform, compliance and legal-ops teams at airlines, banks, insurers and telcos: organisations that already run Db2.

Before release

Wrong policy promises are corrected from the clause text, or held for a specialist, with citations.

In a dispute

An evidence pack: the exact policy version, what the database knew then, and a receipt check.

Around a policy change

Policy PDFs become checked drafts; every answer a change made stale gets a checked correction for its customer.

How it gets adopted: one REST endpoint, a LangChain Runnable, a Langflow component, an MCP tool for agents or a chat widget, in front of any assistant, on OpenRouter, Ollama or IBM watsonx.ai. Runs on Db2 Community Edition; a second domain (a bank fee schedule) reused the schema, check and ledger, adding prompts, data and rule cues. Apache-2.0.

langchain

Langflow

Aurelia Air and Fernhollow Bank are fictional; the airline is modelled on the policy at issue in Moffatt.

Real US DOT refund rules from the eCFR: each question answered by its date's rule (a demonstration).

Expected Impact

Status and next steps

Built and measured

- LangGraph check, two agents, a router; fail closed
- Bitemporal, vector, JSON, SQL PL, RCAC, in-DB AI
- Ledger with receipts; specialist inbox; notices
- Release rule, row permissions and WLM in Db2
- Two domains, an unseen third; 1311 tests
- Live demo under a Db2 spend cap; backtest mode

Next: five of six now done

- Chain head anchored outside Db2, HMAC log
- Per-user API tokens and roles, kept in Db2
- Fewer needless holds: 75.8% to 87.9% ideal
- Names redacted before the LLM and the ledger
- Optional TLS to Db2, tested locally and in CI
- → Cookbook recipe ready to contribute (signed off)

github.com/JustJay7/asof

Jay Malhotra | solo entry

AsOf | Final submission | as of 10 Nov 2026